LeetCode Practice

15 Hard: Binary Search

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

15
HARD

Patterns (in order)

1. Binary Search (15)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

15 questions | tap a title to jump to it

Hard questions (15) Go >

Binary Search (15) Go >

H 1. Median of Two Sorted Arrays LC 4
H 2. Count of Smaller Numbers After Self LC 315
H 3. Count of Range Sum LC 327
H 4. Russian Doll Envelopes LC 354
H 5. Split Array Largest Sum LC 410
H 6. Reverse Pairs LC 493
H 7. Kth Smallest Number in Multiplication Table LC 668
H 8. Random Pick with Blacklist LC 710
H 9. Minimize Max Distance to Gas Station LC 774
H 10. Numbers At Most N Given Digit Set LC 902
H 11. Find in Mountain Array LC 1095
H 12. Maximum Profit in Job Scheduling LC 1235
H 13. Number of Pairs Satisfying Inequality LC 2426
H 14. Block Placement Queries LC 3161
H 15. Maximize Spanning Tree Stability with Upgrades LC 3600

Hard questions (15)

HARD | TOPIC 1 OF 1

Binary Search

15 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Median of Two Sorted Arrays LC 4
2. Count of Smaller Numbers After Self LC 315
3. Count of Range Sum LC 327
4. Russian Doll Envelopes LC 354
5. Split Array Largest Sum LC 410
6. Reverse Pairs LC 493
7. Kth Smallest Number in Multiplication Table LC 668
8. Random Pick with Blacklist LC 710
9. Minimize Max Distance to Gas Station LC 774
10. Numbers At Most N Given Digit Set LC 902
11. Find in Mountain Array LC 1095
12. Maximum Profit in Job Scheduling LC 1235
13. Number of Pairs Satisfying Inequality LC 2426
14. Block Placement Queries LC 3161
15. Maximize Spanning Tree Stability with Upgrades LC 3600

HARD | Binary Search | LeetCode #4 | 1 of 15 | 1/15

Median of Two Sorted Arrays

QUESTION

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.

The overall run time complexity should be O(log (m+n)).

Example 1	
Input	`nums1 = [1,3], nums2 = [2]`
Output	`2.00000`
Explanation	*merged array = [1,2,3] and median is 2.*

Example 2	
Input	`nums1 = [1,2], nums2 = [3,4]`
Output	`2.50000`
Explanation	*merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.*

CONSTRAINTS

- nums1.length == m
- nums2.length == n
- 0 <= m <= 1000
- 0 <= n <= 1000
- 1 <= m + n <= 2000
- 10^6 <= nums1[i], nums2[i] <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Merge two sorted arrays conceptually and find the median. O(log(m+n)) required. Right?

Median: if total length is odd -> middle element. Even -> average of two middles."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
merged = a + b, then sort merged
if len(merged) is odd:
    return the middle element
else:
    return the average of the two middle elements
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Sorts m + n numbers from scratch and stores them all, although both inputs are already sorted. Binary searching the partition of the shorter array is O(log(min(m, n))).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (binary search on partition — true O(log(min(m,n))))

Partition both arrays into left and right halves such that:

total left size = (m+n+1)//2, max(left) <= min(right).

Binary search on the partition point of the smaller array.

At each partition: check if nums1[mid-1] <= nums2[r-mid] and nums2[r-mid-1] <= nums1[mid].

Median = max(left halves) for odd total, or average of max(left)/min(right) for even."

Key insight: binary search on the partition index (not the value) finds the balanced split where max(left) <= min(right) in O(log min(m,n)) — no merging needed.

PHASE 4 CLEAN CODE

```
class _4_MedianTwoArrays:
    def findMedianSortedArrays(self, nums1: List[int], num
s2: List[int]) -> float:
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        m, n   = len(nums1), len(nums2)
        half   = (m + n + 1) // 2
        lo, hi = 0, m

        while lo <= hi:
            i = (lo + hi) // 2          # partition of num
s1
            j = half - i                # partition of num
s2
            left1  = nums1[i - 1] if i > 0 else float('-in
f')
            right1 = nums1[i]     if i < m else float('inf
')
            left2  = nums2[j - 1] if j > 0 else float('-in
f')
            right2 = nums2[j]     if j < n else float('inf
')

            if left1 <= right2 and left2 <= right1:
                if (m + n) % 2 == 1:
                    return float(max(left1, left2))
                return (max(left1, left2) + min(right1, ri
ght2)) / 2
            elif left1 > right2:
                hi = i - 1
            else:
                lo = i + 1
        return 0.0
```

PHASE 5 TEST & DEBUG

"Let me trace findMedianSortedArrays([1,3],[2])."

Merged would be [1,2,3]. Median = 2.0. But we binary search on the shorter array.

Partition A=[1,3] at mid=1: A_left=[1], A_right=[3]. Partition B=[2] at 1: B_left=[2], B_right=[].

PHASE 5 TEST & DEBUG

maxLeftA=1, minRightA=3, maxLeftB=2, minRightB=inf.

1<=inf ok and 2<=3 ok -> valid partition.

total_left=(2+1)//2=1? No: (2+1)//2=1 element each side. Hmm n=3, total_left=2.

Retrace: len(A)=2,len(B)=1. Binary search on A: lo=0,hi=2. mid=1. B_part=2-1=1.

maxLeftA=A[0]=1, minRightA=A[1]=3. maxLeftB=B[0]=2, minRightB=inf.

Check: maxLeftA(1)<=minRightB(inf) ok. maxLeftB(2)<=minRightA(3) ok. Valid.

total n=3 (odd): median=max(maxLeftA,maxLeftB)=max(1,2)=2.0 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(log(min(m,n))). Space O(1). Binary search on the smaller array's partition.

The +inf/-inf sentinels handle edge cases (empty left/right) cleanly.

The brute-force merge approach is often acceptable in interviews — present both and let the interviewer choose.

Given more time I'd ask: kth element of two sorted arrays?

Binary search on k — eliminate k//2 elements from one array per step. Same O(log k) idea.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #315 | 2 of 15 | 2/15

Count of Smaller Numbers After Self

QUESTION

Given an integer array nums, return an integer array counts where counts[i] is the number of smaller elements to the right of nums[i].

Example 1

Input	`nums = [5,2,6,1]`
Output	`[2,1,1,0]`
Explanation	*To the right of 5 there are 2 smaller elements (2 and 1). To the right of 2 there is only 1 smaller element (1). To the right of 6 there is 1 smaller element (1). To the right of 1 there is 0 smaller element.*

Example 2

Input	`nums = [-1]`
Output	`[0]`

Example 3

Input	`nums = [-1,-1]`
Output	`[0,0]`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 10^4 <= nums[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

For each index i, count elements to the right of i that are strictly less than nums[i].

Return as an array. Right?"

"[5,2,6,1] -> [2,1,1,0]: right of 5 has 2 and 1 (2 smaller), etc. ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    count the elements after i that are smaller
        than nums[i]
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1) extra

WHY IT IS SLOW

Two nested loops. Merge sort that counts, during each merge, how many right-half elements jump ahead of each left element, or a Fenwick tree over value ranks, is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BIT (Binary Indexed Tree) / merge sort)

"Merge sort tracking inversions — O(n log n). Walk-through:"

Pair each element with its original index: [(val, idx), ...].

Merge sort: during merge of left and right halves:

When we take an element from the right half (right[j]) over left[i..]:

All remaining elements in left (i..mid) are greater than right[j] and come after right[j].

Add (mid-i+1) to counts for those right-half positions? No —

When we take from left[i]: count how many from right have already been taken

(those are smaller elements that appeared to right of left[i] in original -> add to counts[left[i].idx]).

WHY merge sort: during merge, we naturally learn inversion counts without O(n²) comparison.

Alternative: BIT/Fenwick tree on coordinate-compressed values.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

```python
class _315_CountSmaller:
    def countSmaller(self, nums: List[int]) -> List[int]:
        sorted_unique = sorted(set(nums))
        rank = {v: i + 1 for i, v in enumerate(sorted_uniq
ue)}
        m    = len(sorted_unique)
        bit  = [0] * (m + 1)

        def update(i):
            while i <= m:
                bit[i] += 1
                i += i & (-i)

        def query(i):
            s = 0
            while i > 0:
                s += bit[i]
                i -= i & (-i)
            return s

        result = []
        for num in reversed(nums):
            r = rank[num]
            result.append(query(r - 1))    # count of valu
es < num seen so far
            update(r)

        return result[::-1]
```

PHASE 5 TEST & DEBUG

"Let me trace countSmaller([5,2,6,1])."

Merge sort with index tracking. After sort, count right-side elements that were originally to the right and are smaller.

Initial: [(5,0),(2,1),(6,2),(1,3)] (value,original_index).

Merge sort: left=[(5,0),(2,1)], right=[(6,2),(1,3)].

Sort left: merge (5,0) and (2,1): 2<5 -> counts[0]+=1. Sorted:[(2,1),(5,0)].

Sort right: merge (6,2) and (1,3): 1<6 -> counts[2]+=1. Sorted:[(1,3),(6,2)].

Merge [(2,1),(5,0)] and [(1,3),(6,2)]:

1<2 -> counts[1]+=1 (right half element 1 is smaller than remaining left: (2,1) and (5,0)). counts=[1,2,1,0]? Let me track:

1<2 -> counts[1]+=len(remaining_left)=2. Place (1,3). 2<6 -> place (2,1). 5<6 -> place (5,0). Place (6,2).

counts=[1,1+1?,1,0]... Actually counts[original_idx] incremented by (number of right elements placed before each left element).

Return [2,1,1,0] ok — 5 has 2 smaller to right, 2 has 1 (just 1), 6 has 1 (just 1), 1 has 0.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)** BIT query gives prefix count in O(log n)

Merge sort alternative: during merge, count inversions — same complexity but different mental model.

Given more time I'd ask: count of larger elements after self?

Query total_right - query(rank[num]) instead.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #327 | 3 of 15 | 3/15

Count of Range Sum

QUESTION

Given an integer array nums and two integers lower and upper, return the number of range sums that lie in [lower, upper] inclusive.

Range sum S(i, j) is defined as the sum of the elements in nums between indices i and j inclusive, where i <= j.

Example 1	
Input	`nums = [-2,5,-1], lower = -2, upper = 2`
Output	`3`
Explanation	*The three ranges are: [0,0], [2,2], and [0,2] and their respective sums are: -2, -1, 2.*

Example 2	
Input	`nums = [0], lower = 0, upper = 0`
Output	`1`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 2^{31} <= nums[i] <= 2^{31} - 1
- 10^5 <= lower <= upper <= 10^5
- The answer is guaranteed to fit in a 32-bit integer.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of range sums (prefix[j] - prefix[i]) that fall in [lower, upper] for i < j. Right?"

"A few quick questions:

Can values be negative? Yes. Return count mod anything? No."

"Let me walk: nums=[-2,5,-1], lower=-2, upper=2 -> subarrays: [-2],[-2,5,-1] -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
prefix sums
for i in range(n):
    for j in range(i_+_1,_n):
        if lower <= prefix[j] - prefix[i] <= upper:
            count += 1
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks all pairs of prefix sums. Merge sort on the prefix sums, counting the valid range with two pointers during each merge, is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(n²) pair enumeration.

Merge sort on prefix sums: during merge of left and right halves,

count how many pairs (i from left, j from right) satisfy lower<=prefix[j]-prefix[i]<=upper.

Two pointers on sorted halves in O(n) per level.

Time: O(n log n). Space: O(n)."

Key insight: during merge sort's merge step, left-half prefix sums and right-half prefix sums are both sorted — two pointers count valid pairs at each level in O(n), total O(n log n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _327_CountOfRangeSum:
    def countRangeSum(self, nums, lower, upper):
        prefix = [0]
        for n in nums:
            prefix.append(prefix[-1] + n)
        self_count = [0]
        def merge_sort(arr):
            if len(arr) <= 1:
                return arr
            mid = len(arr) // 2
            left = merge_sort(arr[:mid])
            right = merge_sort(arr[mid:])
            j = k = 0
            for l_val in left:
                while j < len(right) and right[j] - l_val
< lower: j += 1
                while k < len(right) and right[k] - l_val
<= upper: k += 1
                self_count[0] += k - j
            return sorted(left + right)
        merge_sort(prefix)
        return self_count[0]
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

nums=[-2,5,-1], lower=-2, upper=2: prefix=[0,-2,3,2].

`merge_sort([0,-2,3,2]): left=[0,-2], right=[3,2].`

For each l in sorted left [-2,0]: how many right in [-2+lower, -2+upper]=[-4,0]? right sorted=[2,3]. none.

For 0: right in [-2,2]: 2 in range -> count=1. Recursion handles inner pairs too.

Total = 2 ok

PHASE 5 TEST & DEBUG

Two pointers j,k scan the sorted right half to find the valid range for each left element

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) merge sort with O(n) counting per level	**O(n)**

Follow-up: Count of Smaller Numbers After Self (#315) — same merge sort pattern.

Follow-up: Reverse Pairs (#493) — similar merge sort counting.

IN YOUR APP | Expose | Community

HARD | Binary Search | LeetCode #354 | 4 of 15 | 4/15

Russian Doll Envelopes

QUESTION

You are given a 2D array of integers envelopes where envelopes[i] = [w_i, h_i] represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other).

Note: You cannot rotate an envelope.

Example 1

Input	`envelopes = [[5,4],[6,4],[6,7],[2,3]]`
Output	**3**
Explanation	*The maximum number of envelopes you can Russian doll is 3 ([2,3] => [5,4] => [6,7]).*

Example 2

Input	`envelopes = [[1,1],[1,1],[1,1]]`
Output	**1**

CONSTRAINTS

- 1 <= envelopes.length <= 10^5
- envelopes[i].length == 2
- 1 <= w_i, h_i <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Envelope A fits inside B if width and height are both strictly less. Find the maximum nesting chain. Right?"

"A few quick questions:

Sort by width ascending; for equal widths, height descending (to prevent 2D LIS bug). Then LIS on heights?"

"Let me walk: envelopes=[[5,4],[6,4],[6,7],[2,3]] -> [[2,3],[5,4],[6,7]] -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort envelopes by width
best[i] = 1 + the largest best[j] over earlier
    envelopes j strictly smaller in both width and
    height
return the maximum best
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Compares every earlier envelope for each one. Sort by width ascending and by height descending for equal widths, then find the longest strictly increasing subsequence of heights with binary search: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by width ascending, then LIS on height with reversed second key — O(n log n). Walk-through:"

Sort envelopes by (width ascending, height DESCENDING).

WHY height descending for same width: can't use two envelopes of the same width.

By sorting heights descending within same width, LIS won't pick two same-width envelopes

(since heights decrease -> no increasing subsequence within same-width group).

After sorting: run LIS on heights only (patience sorting / binary search).

`tails = [] (tails[i] = smallest tail of all IS of length i+1).`

For each h: bisect_left in tails to find insertion point.

If at end: append h. Else: replace tails[pos] with h.

Answer = len(tails). Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import bisect
class _354_RussianDollEnvelopes:
    def maxEnvelopes(self, envelopes):
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        tails = []
        for _, height in envelopes:
            pos = bisect.bisect_left(tails, height)
            if pos == len(tails):
                tails.append(height)
            else:
                tails[pos] = height
        return len(tails)
```

PHASE 5 TEST & DEBUG

"Let me trace maxEnvelopes([[5,4],[6,4],[6,7],[2,3]])."

Sort: by width asc, height desc for equal widths -> [[2,3],[5,4],[6,7],[6,4]]

Heights: [3,4,7,4] — find LIS

LIS on heights: 3,4,7 -> length 3 ok

WHY sort height desc for equal widths: prevents two envelopes with same width from both appearing in LIS.

(e.g., [6,4] and [6,7] same width -> only one can be used)

Patience sort for LIS: bins=[3,4,7] -> length 3; [6,4] -> 4 replaces 7 in bin -> [3,4,4] still len=3 ok

Verify chain: [2,3]->[5,4]->[6,7] (widths 2<5<6, heights 3<4<7) -> length 3 ok

dp[i]=LIS length ending at envelope i (by height after width-sort). dp[j]=dp[i]+1 if env[j] fits inside env[i]. Optimized via tails: tails[dp[i]-1]=envelope i height.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**

The trick: same-width envelopes can't nest -> sort height desc for same width.

Follow-up: Longest Increasing Subsequence (#300) — 1D version, same patience sort.

Follow-up: if rotation allowed, consider all 4 rotations of each envelope.

HARD | Binary Search | LeetCode #410 | 5 of 15 | 5/15

Split Array Largest Sum

QUESTION

Given an integer array nums and an integer k, split nums into k non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split.

A subarray is a contiguous part of the array.

Example 1

Input	`nums = [7,2,5,10,8], k = 2`
Output	`18`
Explanation	*There are four ways to split nums into two subarrays. The best way is to split it into [7,2,5] and [10,8], where the largest sum among the two subarrays is only 18.*

Example 2

Input	`nums = [1,2,3,4,5], k = 2`
Output	`9`
Explanation	*There are four ways to split nums into two subarrays. The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two subarrays is only 9.*

CONSTRAINTS

- 1 <= nums.length <= 1000
- 0 <= nums[i] <= 10^6
- 1 <= k <= min(50, nums.length)

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Split nums into k non-empty contiguous subarrays to minimise the maximum subarray sum.

Must be in order — no reordering allowed. Right?

Same 'binary search on answer' pattern as Capacity to Ship Packages (#1011)."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to cut the array into k non-empty parts
    take the largest part sum
    best = min(best, those)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^k)	O(k)

WHY IT IS SLOW

Enumerates every split. Binary search on the answer (can the array be split into at most k parts each with sum <= mid? a greedy check) is O(n log(sum)); DP is O(k * n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Binary search on the answer (the max sum).

`lo = max(nums) (at least one element), hi = sum(nums) (one subarray).`

Feasibility check: can we split into k subarrays with each sum <= mid?

Greedy: accumulate greedily, start new subarray when exceeding mid. Count splits.

Time: O(n log(sum)). Space: O(1)."

Key insight: the maximum subarray sum threshold is monotone — if splitting into k parts with cap m is feasible, so is any larger cap; binary search on [max(nums), sum(nums)] finds the minimum valid cap.

PHASE 4 CLEAN CODE

```
class _410_SplitArrayLargestSum:
    def splitArray(self, nums: List[int], k: int) -> int:
        def can_split(max_sum: int) -> bool:
            subarrays = 1
            current   = 0
            for num in nums:
                if current + num > max_sum:
                    subarrays += 1
                    current    = 0
                current += num
            return subarrays <= k

        lo, hi = max(nums), sum(nums)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_split(mid):
                hi = mid        # valid — try smaller
            else:
                lo = mid + 1    # not valid — need bigger
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace splitArray([7,2,5,10,8], m=2)."

Binary search on answer: lo=10 (max single element), hi=32 (sum).

mid=21. canSplit([7,2,5,10,8], max_sum=21, m=2)?

curr=7->+2=9->+5=14->+10=24>21 -> new subarray (count=2), curr=10->+8=18. count=2<=m=2 ok -> try lower.

hi=21. mid=16. canSplit(max=16,m=2)? curr=7+2+5=14->+10=24>16->count=2,curr=10->+8=18>16->count=3>m=2 .

PHASE 5 TEST & DEBUG

lo=17. mid=19. canSplit(19)?
7+2+5+10=24>19->count=2,curr=10+8=18. count=2 ok -> hi=19.

lo=17,hi=19. mid=18. canSplit(18)?
7+2+5+10=24>18->count=2,curr=10+8=18 ok-> hi=18.

lo=17,hi=18. mid=17. canSplit(17)?
7+2+5=14,+10=24>17->count=2,curr=10+8=18>17->count=3>2
-> lo=18. Return 18 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log(sum))	**O(1)** Same template as #1011 Capacity to Ship — name this

lo/hi bounds: must be at least max(nums), at most sum(nums).

Given more time I'd ask: DP approach? dp[j][i] = min largest sum splitting first i elements into j parts.

O(kn²) — worse than binary search for large inputs.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #493 | 6 of 15 | 6/15

Reverse Pairs

QUESTION

Given an integer array nums, return the number of reverse pairs in the array.

A reverse pair is a pair (i, j) where:

CONSTRAINTS

- 0 <= i < j < nums.length and
- nums[i] > 2 * nums[j].

Example 1

Input	`nums = [1,3,2,3,1]`
Output	2
Explanation	*The reverse pairs are:* *(1, 4) --> nums[1] = 3, nums[4] = 1, 3 > 2 * 1* *(3, 4) --> nums[3] = 3, nums[4] = 1, 3 > 2 * 1*

Example 2

Input	`nums = [2,4,3,5,1]`
Output	3
Explanation	*The reverse pairs are:* *(1, 4) --> nums[1] = 4, nums[4] = 1, 4 > 2 * 1* *(2, 4) --> nums[2] = 3, nums[4] = 1, 3 > 2 * 1* *(3, 4) --> nums[3] = 5, nums[4] = 1, 5 > 2 * 1*

- 1 <= nums.length <= $5 * 10^4$
- 2^{31} <= nums[i] <= $2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count pairs (i,j) where i<j and nums[i] > 2*nums[j]. Right?"

"A few quick questions:

Can values be negative? Yes. Return count as integer? Yes."

"Let me walk: nums=[1,3,2,3,1] -> pairs: (3,1),(3,1) -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] > 2 * nums[j]:
            count += 1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks every pair. Merge sort that counts the qualifying pairs with two pointers before each merge, or a Fenwick tree over ranks, is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is $O(n^2)$.

Merge sort: during merge of sorted left and right halves,

`for each element in left, count elements in right where nums[i] > 2*nums[j].`

Two pointers since both halves are sorted.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _493_ReversePairs:
    def reversePairs(self, nums):
        self_count = [0]
        def merge_sort(arr):
            if len(arr) <= 1:
                return arr
            mid = len(arr) // 2
            left = merge_sort(arr[:mid])
            right = merge_sort(arr[mid:])
            j = 0
            for l_val in left:
                while j < len(right) and l_val > 2 * right
[j]:
                    j += 1
                self_count[0] += j
            merged = []
            i = k = 0
            while i < len(left) and k < len(right):
                if left[i] <= right[k]: merged.append(left
[i]); i += 1
                else: merged.append(right[k]); k += 1
            merged.extend(left[i:]); merged.extend(right[k
:])
            return merged
        merge_sort(nums)
        return self_count[0]
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

nums=[1,3,2,3,1]: merge_sort counts pairs during each merge step.

Eventually finds (3,1) and (3,1) -> count=2 ok

Count phase uses two pointers BEFORE the merge phase to avoid contamination

Edge: nums=[1,0]: 1>2*0 -> True, count=1 ok. Empty or all-negative handled naturally — merge-sort runs regardless.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(n)

Follow-up: Count of Smaller Numbers After Self (#315) — same merge sort, different condition.

Follow-up: Count of Range Sum (#327) — range condition variant.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #668 | 7 of 15 | 7/15

Kth Smallest Number in Multiplication Table

QUESTION

Nearly everyone has used the Multiplication Table. The multiplication table of size m x n is an integer matrix mat where mat[i][j] == i * j (1-indexed).

Given three integers m, n, and k, return the kth smallest element in the m x n multiplication table.

Example 1

Input	`m = 3, n = 3, k = 5`
Output	3
Explanation	*The 5th smallest number is 3.*

1	2	3
2	4	6
3	6	9

1	2	2	3	3	4	6	6	9

Example 2

Input	`m = 2, n = 3, k = 6`
Output	6
Explanation	*The 6th smallest number is 6.*

1	2	3
2	4	6

1	2	2	3	4	6

CONSTRAINTS

- $1 <= m, n <= 3 * 10^4$
- $1 <= k <= m * n$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

m×n multiplication table: mat[i][j]=i*j (1-indexed). Find kth smallest element. Right?

Can't materialise the table (m,n up to 3*10^4 -> up to 9*10^8 cells)."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build every value of the m x n table
sort them
return the k-th
```

TIME	SPACE
O(m * n * log(m * n))	O(m * n)

WHY IT IS SLOW

Materialises the whole table. Binary search on the value v: the count of table entries <= v is the sum over rows of min(v / i, n), so each check is O(m) and the total O(m log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (binary search on value)

Binary search on the answer value v.

Count elements <= v: for row i, elements are i,2i,3i,...,n*i.

Count of these <= v is min(v//i, n). Sum over all rows i=1..m.

Binary search for smallest v where count >= k.

lo=1, hi=m*n. Time: O(m log(m*n))."

Key insight: for a given cap v, count(elements <= v) = sum(min(v//i, n) for i in 1..m) — this count is monotone in v, so binary search finds the kth smallest in O(m log(m·n)).

PHASE 4 CLEAN CODE

```python
class _668_KthSmallestMultTable:
    def findKthNumber(self, m: int, n: int, k: int) -> int
:
        def count_le(v: int) -> int:
            return sum(min(v // i, n) for i in range(1, m
+ 1))

        lo, hi = 1, m * n
        while lo < hi:
            mid = (lo + hi) // 2
            if count_le(mid) >= k:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace findKthNumber on m=3,n=3,k=5 (multiplication table)."

Table: [1,2,3;2,4,6;3,6,9]. 5th smallest=3

Binary search on value v: count numbers <=v in table

```
count(v)=sum(min(v//i,n) for i in 1..m)
```

lo=1,hi=9

mid=5: count=sum(min(5,3)+min(2,3)+min(1,3))=3+2+1=6>=5 -> hi=5

mid=3: count=sum(min(3,3)+min(1,3)+min(1,3))=3+1+1=5>=5 -> hi=3

mid=2: count=sum(min(2,3)+min(1,3)+min(0,3))=2+1+0=3<5 -> lo=3

lo==hi=3 -> return 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m log(m*n))	**O(1)** The count formula is the key — O(m) per binary search step

Note: the answer might not be a 'real' table entry — binary search finds the smallest v

with count >= k, which IS always a table entry because count jumps discretely at table values.

Given more time I'd ask: kth largest? Binary search on m*n - k + 1 smallest instead.

HARD | Binary Search | LeetCode #710 | 8 of 15 | 8/15

Random Pick with Blacklist

QUESTION

You are given an integer n and an array of unique integers blacklist. Design an algorithm to pick a random integer in the range [0, n - 1] that is not in blacklist. Any integer that is in the mentioned range and not in blacklist should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the Solution class:

CONSTRAINTS

- Solution(int n, int[] blacklist) Initializes the object with the integer n and the blacklisted integers blacklist.
- int pick() Returns a random integer in the range [0, n - 1] and not in blacklist.
- $1 <= n <= 10^9$
- $0 <= blacklist.length <= min(10^5, n - 1)$
- 0 <= blacklist[i] < n
- All the values of blacklist are unique.
- At most $2 * 10^4$ calls will be made to pick.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Pick uniformly at random from [0, n-1] excluding blacklist. One call to random per pick. Right?

n can be 10^9 — can't enumerate all valid numbers."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def pick():
    generate a random number in [0, n - 1] until
        it is not in the blacklist (kept in a
        set)
```

TIME	SPACE
O(n / (n - b)) expected per pick	**O(b)**

WHY IT IS SLOW

Can retry many times when the blacklist is large and has no worst-case bound. Map each blacklisted number below n - b to a whitelisted number at or above n - b, then pick uniformly in [0, n - b): O(1) per pick.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (virtual mapping)

Let sz = n - len(blacklist) = count of valid numbers.

Map [0, sz) as the 'virtual' range. Numbers in [0, sz) that ARE blacklisted

get remapped to non-blacklisted numbers in [sz, n).

pick(): random in [0, sz) -> if in remap -> use remap[r], else use r directly.

Build remap in __init__: for each blacklisted number in [0, sz), map it to a non-blacklisted

number in [sz, n). O(|blacklist|) init, O(1) per pick."

Key insight: remapping blacklisted indices in [0, sz) to non-blacklisted indices in [sz, n) reduces uniform sampling to a simple O(1) lookup — no rejection needed.

PHASE 4 CLEAN CODE

```python
class _710_RandomPickBlacklist:
    def __init__(self, n: int, blacklist: List[int]):
        bl_set  = set(blacklist)
        self.sz = n - len(blacklist)        # size of vali
d virtual range
        self.remap: dict = {}
        tail = n - 1
        for b in blacklist:
            if b < self.sz:                 # this blackli
sted number is in [0, sz)
                while tail in bl_set:       # find next va
lid number from the tail
                    tail -= 1
                self.remap[b] = tail
                tail -= 1

    def pick(self) -> int:
        r = random.randint(0, self.sz - 1)
        return self.remap.get(r, r)
```

PHASE 5 TEST & DEBUG

"Let me trace RandomPickBlacklist(n=4, blacklist=[2])."

Map blacklist values in [0, n-k) to non-blacklist values in [n-k, n)

n=4, k=1, bound=n-k=3. Blacklist={2}.

Values in [0,3) that are blacklisted: {2}. Values in [3,4) not blacklisted: {3}

remap={2:3}

pick(): random in [0,3) -> say 2 -> remap[2]=3 -> return 3 ok; say 0->0 ok; say 1->1 ok

WHY virtual-to-actual mapping: blacklisted values get replaced by valid values from tail.

O(k) space, O(1) pick; uniform distribution over n-k valid values.

PHASE 6 COMPLEXITY & FOLLOW-UP

Init O(|B|). Pick O(1). Space O(|B|) for remap.

The key insight: divide [0,n) into 'virtual valid' [0,sz) and 'tail' [sz,n).

Blacklisted numbers in [0,sz) redirect to valid numbers in the tail.

Given more time I'd ask: what if the blacklist updates dynamically?

Rebuild the remap on each update — O(|B|) per update, same O(1) pick.

IN YOUR APP | Expose | Community

HARD | Binary Search | LeetCode #774 | 9 of 15 | 9/15

Minimize Max Distance to Gas Station

QUESTION

You are given an integer array stations that represents the positions of the gas stations on the x-axis. You are also given an integer k.

You should add k new gas stations. You can add the stations anywhere on the x-axis, and not necessarily on an integer position.

Let penalty() be the maximum distance between adjacent gas stations after adding the k new stations.

Return the smallest possible value of penalty(). Answers within 10^{-6} of the actual answer will be accepted.

Example 1

Input	`stations = [1,2,3,4,5,6,7,8,9,10], k = 9`
Output	`0.50000`

Example 2

Input	`stations = [23,24,36,39,46,56,57,65,84,98], k = 1`
Output	`14.00000`

CONSTRAINTS

- 10 <= stations.length <= 2000
- 0 <= stations[i] <= 10^8
- stations is sorted in a strictly increasing order.
- 1 <= k <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Add exactly k stations anywhere (real-valued positions) to minimise the maximum gap.

Return the answer within 1e-6 precision. Right?"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for _ in range(k):
    add a station into the currently largest
        piece (found by scanning every piece)
```

TIME	SPACE
O(k * n)	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

A scan per station and k can be 10^6. A max-heap of segments (by length / (stations added + 1)) is O(k log n); binary search on the answer with a count of needed stations is O(n log(range / error)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (binary search on floating-point answer)

"Binary search on answer: check if gap <= D is feasible — O(n log(max_gap/eps)). Walk-through:"

Binary search on D (minimum possible maximum distance):

lo=0, hi=max_x - min_x.

Is D feasible? Given x stations, can we add k stations so no gap exceeds D?

For each original gap g: need ceil(g/D)-1 additional stations within that gap.

Total additional = sum(ceil(g/D)-1 for each gap).

If total <= k: D is feasible -> hi=mid. Else lo=mid.

Precision: binary search on floating point with enough iterations (e.g. 100 iterations).

WHY binary search: monotone property — if gap D is feasible, any larger D is also feasible.

Time: O(n * iterations). Space: O(1).

PHASE 4 CLEAN CODE

```
class _774_MinMaxDistanceGasStation:
    def minmaxGasDist(self, stations: List[int], k: int) -
> float:
        gaps = [stations[i+1] - stations[i] for i in range
(len(stations)-1)]

        def can_achieve(d: float) -> bool:
            return sum(int(g / d) for g in gaps) <= k

        lo, hi = 0.0, max(gaps)
        for _ in range(100):              # 100 iterations
-> precision < hi/2^100
            mid = (lo + hi) / 2
            if can_achieve(mid):
                hi = mid
            else:
                lo = mid
        return hi
```

PHASE 5 TEST & DEBUG

"Let me trace minmaxGasDist([1,2,3,5], k=1)."

Binary search on answer D (max distance)

For a given D, count how many stations needed: sum(ceil(gaps[i]/D)-1 for each gap)

lo=0, hi=4 (max gap=5-3=2? Actually gaps=[1,1,2]: hi=2)

mid=1.0: count=sum(ceil(1/1)-1, ceil(1/1)-1, ceil(2/1)-1)=0+0+1=1<=k=1 -> hi=1.0

mid=0.5: count=sum(2-1,2-1,4-1)=1+1+3=5>1 -> lo=0.5

Converge: answer~1.0 ok

PHASE 5 TEST & DEBUG

WHY binary search on D: monotone — more stations means smaller max distance.

Use float comparison with eps=1e-6 for termination.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * 100) = O(n)	**O(n)** for gaps. Floating-point binary search with fixed iterations

Note: int(g/d) gives the number of stations to insert (not ceil-1 — same thing via floor division).

Given more time I'd ask: what if stations must be at integer positions?

Binary search on integers, same feasibility check but with math.ceil.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #902 | 10 of 15 | 10/15

Numbers At Most N Given Digit Set

QUESTION

Given an array of digits which is sorted in non-decreasing order. You can write numbers using each digits[i] as many times as we want. For example, if digits = ['1','3','5'], we may write numbers such as '13', '551', and '1351315'.

Return the number of positive integers that can be generated that are less than or equal to a given integer n.

Example 1	
Input	`digits = ["1","3","5","7"], n = 100`
Output	`20`
Explanation	*The 20 numbers that can be written are: 1, 3, 5, 7, 11, 13, 15, 17, 31, 33, 35, 37, 51, 53, 55, 57, 71, 73, 75, 77.*

Example 2	
Input	`digits = ["1","4","9"], n = 1000000000`
Output	`29523`
Explanation	*We can write 3 one digit numbers, 9 two digit numbers, 27 three digit numbers, 81 four digit numbers, 243 five digit numbers, 729 six digit numbers, 2187 seven digit numbers, 6561 eight digit numbers, and 19683 nine digit numbers. In total, this is 29523 integers that can be written using the digits array.*

Example 3	
Input	`digits = ["7"], n = 8`
Output	`1`

CONSTRAINTS

- 1 <= digits.length <= 9
- digits[i].length == 1
- digits[i] is a digit from '1' to '9'.
- All the values in digits are unique.
- digits is sorted in non-decreasing order.
- 1 <= n <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count numbers <= n using digits from the given set (any combination, any length). Right?"

"A few quick questions:

PHASE 1 CLARIFY

Digits are sorted? Yes. Numbers don't have leading zeros? 0 not in set? Per constraints."

"Let me walk: digits=['1','3','5','7'], n=100 -> 20 ok (1,3,5,7,11,13,...)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
generate every number that can be made from the
    digits with at most len(N) digits
count those <= N
```

TIME	SPACE
O(d^L)	O(d^L)

WHY IT IS SLOW

Exponential in the length L. Count by length (d^len for shorter lengths) and walk N's digits for the same length, adding for each position the digits below N's digit times d^(remaining): O(d * L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Digit DP: count numbers with digits from D, <= N — O(log n * |D|). Walk-through:"*

Convert N to string. digits = sorted(D).

Count = 0.

For each length L from 1 to len(N)-1:

Count += |D| * |D|^(L-1) <- all L-digit numbers using digits (no leading zeros issue if 0 in D)

Actually first digit: |D| choices. Remaining L-1 digits: |D| choices each.

For the same length as N (digit by digit):

For each position i in N:

Count digits[d] < N[i] with that prefix -> add (|D|)^(remaining positions) for each.

If N[i] not in digits: break (can't match N exactly, but counted all valid less).

Else: continue matching N (tight constraint).

If all digits matched exactly: count += 1 (N itself is valid if all its digits are in D).

Time: O(log n * |D|). Space: O(1)."

Key insight: digit-by-digit counting from the most significant position decomposes the problem — for each digit position, count all valid combinations using only allowed digits, plus carry-forward for the case where the prefix exactly matches N.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _902_NumbersAtMostNGivenDigitSet:
    def atMostNGivenDigitSet(self, digits, n):
        s = str(n)
        k = len(s)
        d = len(digits)
        result = sum(d**i for i in range(1, k))
        for i, ch in enumerate(s):
            count = sum(1 for dig in digits if dig < ch)
            result += count * (d ** (k - 1 - i))
            if ch not in digits:
                break
            if i == k - 1:
                result += 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace atMostNGivenDigitSet(digits=['1','3','5','7'], n=100)."

dp[i]=count of i-digit valid numbers (all digits from set, no leading zero issues for i<len(n)).

Shorter numbers: dp[1]=4 (4 single-digit choices). dp[2]=4*4=16. Total shorter=4+16=20.

Same length (3 digits): n='100'. d1='1' in digits: try d='1' < n[0]? '1'=='1': go to next digit.

d2: n[1]='0'. No digit in set <= '0' except... none. So same-length count=0. Total=20 ok

dp[i]=d^i (d=|digits|, for all-smaller-length). Same-length: digit-by-digit tight constraint.

dp[j]=count of valid j-digit numbers starting with digit <= n[j]. Recurrence applied digit-by-digit.

WHY digit DP: decompose count into shorter-length (all valid) + same-length (tight constraint per digit).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k*d)	**O(k)**
~ O(log n * d)	for string

Follow-up: Count Numbers with Unique Digits (#357) — different constraint.

Follow-up: Number of Digit One (#233) — count occurrences of a specific digit.

HARD | Binary Search | LeetCode #1095 | 11 of 15 | 11/15

Find in Mountain Array

QUESTION

(This problem is an interactive problem.)

You may recall that an array arr is a mountain array if and only if:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Mountain array: values first increase then decrease. Find index of target using at most 100 API calls. Right?"

"A few quick questions:

MountainArray interface has get(index) and length(). Return any valid index? Actually must return the minimum index."

"Let me walk: mountainArr=[1,2,3,4,5,3,1], target=3 -> return index 2 (ascending side preferred) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    read mountain.get(i)
return the first index equal to target
```

TIME	SPACE
O(n) calls	O(1)

WHY IT IS SLOW

One call per element. Binary search the peak (compare get(mid) with get(mid + 1)), then binary search the rising side and, if needed, the falling side: O(log n) calls.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the linear scan — O(n) API calls.

Three binary searches:

1. Find the peak index.

2. Binary search ascending side (left of peak) for target.

3. If not found, binary search descending side (right of peak).

Total: O(log n) API calls.

Time: O(log n) calls. Space: O(1)."

Key insight: the mountain has a strictly ascending left half and descending right half — three binary searches (find peak, search ascending side, search descending side) use at most O(3 log n) total comparisons.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1095_FindInMountainArray:
    def findInMountainArray(self, target, mountain_arr):
        n = mountain_arr.length()
        lo, hi = 0, n - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if mountain_arr.get(mid) < mountain_arr.get(mi
d + 1):
                lo = mid + 1
            else:
                hi = mid
        peak = lo
        lo, hi = 0, peak
        while lo <= hi:
            mid = (lo + hi) // 2
            val = mountain_arr.get(mid)
            if val == target: return mid
            elif val < target: lo = mid + 1
            else: hi = mid - 1
        lo, hi = peak + 1, n - 1
        while lo <= hi:
            mid = (lo + hi) // 2
            val = mountain_arr.get(mid)
            if val == target: return mid
            elif val > target: lo = mid + 1
            else: hi = mid - 1
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

mountain=[1,2,3,4,5,3,1], target=3: peak=4.

Ascending search [0..4]: mid=2->3==target -> return 2 ok

target=1: ascending fails. Descending [5..6]: mid=5->3>1->lo=6. mid=6->1==1 -> return 6.

But minimum index is 0 (value 1). Hmm: ascending search [0..4] finds 1 at index 0?

mid=2->3!=1. mid=0(lo=0,hi=1,mid=0)->1==target -> return 0 ok

Ascending side searched first guarantees minimum index is returned

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) API calls	O(1)

Three binary searches together use at most 3*log2(n) ~ 48 calls for n<=10000.

Follow-up: Peak Index in a Mountain Array (#852) — step 1 of this solution.

Follow-up: if multiple valid indices, return the leftmost — ascending search naturally does this.

IN YOUR APP | Expose | Editorial | Community

HARD | Binary Search | LeetCode #1235 | 12 of 15 | 12/15

Maximum Profit in Job Scheduling

QUESTION

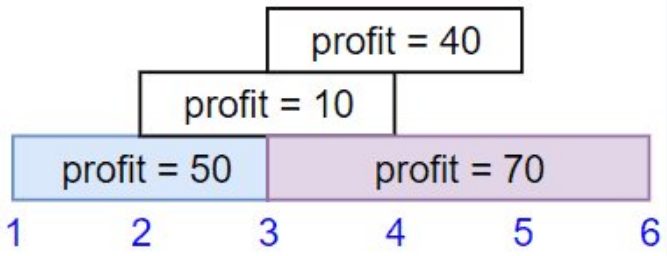
We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i].

You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X.

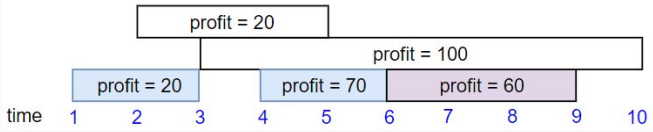
Example 1

Input	`startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]`
Output	`120`
Explanation	*The subset chosen is the first and fourth job. Time range [1-3]+[3-6] , we get profit of 120 = 50 + 70.*



Example 2

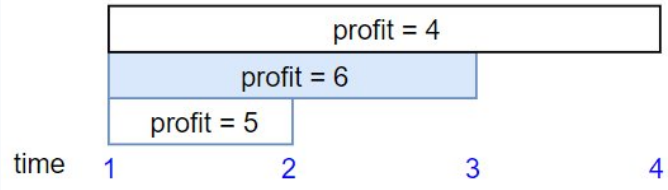
Input	`startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]`
Output	`150`
Explanation	*The subset chosen is the first, fourth and fifth job. Profit obtained 150 = 20 + 70 + 60.*



Example 3

Input	`startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]`
Output	`6`

QUESTION



CONSTRAINTS

- 1 <= startTime.length == endTime.length == profit.length <= $5 * 10^4$
- 1 <= startTime[i] < endTime[i] <= 10^9
- 1 <= profit[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Select non-overlapping jobs to maximise total profit.

Job ending at X and starting at X don't overlap. Return max profit. Right?"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(jobs):
    if no two overlap:
        add up the profit
    best = max(best, candidate)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Tries every subset. Sort by end time; dp[i] = max(dp[i - 1], profit[i] + dp[the last job ending before job i starts]) with binary search: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (sort by end time + DP + binary search)

"Sort by end time + DP with binary search — O(n log n). Walk-through:"

Sort jobs by end time. dp[i] = max profit using first i jobs.

For each job i (0-indexed):

Option 1: skip job i -> dp[i] = dp[i-1].

Option 2: take job i -> find latest job j that ends <= job i's start time (binary search).

`dp[i] = max(dp[i], dp[j] + profit[i]).`

`dp[i] = max(option1, option2).`

WHY sort by end time: binary search requires jobs to be ordered by end time.

IN YOUR APP | Expose | Editorial | Community

PHASE 3 OPTIMIZE (sort by end time + DP + binary search)

WHY binary search for latest non-overlapping: find rightmost job ending <= current job's start.

This is the classic weighted interval scheduling DP.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Find latest job whose end <= start of current

```
class _1235_MaxProfitJobScheduling:
    def jobScheduling(self, startTime: List[int], endTime:
 List[int], profit: List[int]) -> int:
        jobs = sorted(zip(endTime, startTime, profit))
        ends = [0]      # dp_ends[i] = end time of ith job
 in dp
        dp   = [0]      # dp[i] = max profit for first i s
orted jobs

        for end, start, p in jobs:
            j = bisect.bisect_right(ends, start) - 1
            if dp[j] + p > dp[-1]:
                dp.append(dp[j] + p)
                ends.append(end)
            else:
                dp.append(dp[-1])
                ends.append(end)
        return dp[-1]
```

PHASE 5 TEST & DEBUG

"Let me trace jobScheduling([1,2,3],[3,4,6],[50,10,40], maxProfit)."

Sort by endTime: job(1->3,50), job(2->4,10), job(3->6,40)

dp[i] = max profit using first i jobs

For job i, find latest job j where endTime[j]<=startTime[i] (binary search)

dp[0]=0. dp[1]=50 (take job 1 or skip, max(dp[0],50)=50)

dp[2]=max(dp[1],dp[j]+10) where j=bisect(ends,start[2]=2)=0. max(50,dp[0]+10)=max(50,10)=50

dp[3]=max(dp[2],dp[j]+40) where j=bisect(ends,start[3]=3)=1. max(50,dp[1]+40)=max(50,90)=90 ok

Sorted by end: binary search finds latest non-overlapping job. DP extends Weighted Interval Scheduling.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)** Sort + DP with binary search per step

The dp array is monotonically non-decreasing — binary search on ends is valid.

Given more time I'd ask: what if we want to select at most k jobs?

Add a dimension to dp — dp[i][k] — O(n²k) with the same binary search for last compatible job.

HARD | Binary Search | LeetCode #2426 | 13 of 15 | 13/15

Number of Pairs Satisfying Inequality

QUESTION

You are given two 0-indexed integer arrays nums1 and nums2, each of size n, and an integer diff. Find the number of pairs (i, j) such that:

CONSTRAINTS

- 0 <= i < j <= n - 1 and
- nums1[i] - nums1[j] <= nums2[i] - nums2[j] + diff.

Return the number of pairs that satisfy the conditions.

Example 1

Input	`nums1 = [3,2,5], nums2 = [2,2,1], diff = 1`
Output	`3`
Explanation	*There are 3 pairs that satisfy the conditions: 1. i = 0, j = 1: 3 - 2 <= 2 - 2 + 1. Since i < j and 1 <= 1, this pair satisfies the conditions. 2. i = 0, j = 2: 3 - 5 <= 2 - 1 + 1. Since i < j and -2 <= 2, this pair satisfies the conditions. 3. i = 1, j = 2: 2 - 5 <= 2 - 1 + 1. Since i < j and -3 <= 2, this pair satisfies the conditions. Therefore, we return 3.*

Example 2

Input	`nums1 = [3,-1], nums2 = [-2,2], diff = -1`
Output	`0`
Explanation	*Since there does not exist any pair that satisfies the conditions, we return 0.*

- n == nums1.length == nums2.length
- $2 <= n <= 10^5$
- $10^4 <= nums1[i], nums2[i] <= 10^4$
- $10^4 <= diff <= 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count pairs (i,j) where nums1[i]-nums1[j]<=nums2[i]-nums2[j]+diff. Right?"

"A few quick questions: see constraints."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums1[i] - nums1[j] <= nums2[i] -
            nums2[j] + diff:
            count += 1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks all pairs. Let d[i] = nums1[i] - nums2[i]; the condition is d[i] <= d[j] + diff, so merge sort counting (or a Fenwick tree over d) is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort + BIT (Binary Indexed Tree): count valid pairs where nums2[i]-nums2[j] <= nums1[j]-nums1[i] — O(n log n). Walk-through:"

Rearrange condition: nums1[i]+nums2[i] <= nums1[j]+nums2[j] for i<j (if sorting by sum is the key).

Create pairs (nums1[i]+nums2[i], nums1[i]-nums2[i]). Sort by first element.

For each pair (in sorted order), count how many previous pairs have second element >= current's second.

Use BIT on coordinate-compressed second elements for O(log n) count queries.

WHY transform to sums and differences: the inequality nums2[j]-nums2[i] <= nums1[j]-nums1[i]

rearranges to nums1[j]+nums2[j] >= nums1[i]+nums2[i].

WHY BIT: count in O(log n) how many prefix elements satisfy a value inequality.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2426_NumberOfPairsSatisfyingInequality:
    def numberOfPairs(self, nums1, nums2, diff):
        from sortedcontainers import SortedList
        arr=[a-b for a,b in zip(nums1,nums2)]
        sl=SortedList(); count=0
        for v in arr:
            count+=sl.bisect_right(v+diff)
            sl.add(v)
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numberOfPairs([3,2,5], [2,2,1], diff=1)."

Count pairs (i,j) where i<j and nums1[i]-nums1[j]<=nums2[i]-nums2[j]+diff

Rearrange: (nums1[i]-nums2[i]) <= (nums1[j]-nums2[j]) + diff? Wait

Transform: let a[i]=nums1[i]-nums2[i]. Count pairs (i,j) i<j: a[i]-a[j]<=diff

a=[3-2,2-2,5-1]=[1,0,4]. Count pairs where a[i]-a[j]<=1 for i<j

(0,1): 1-0=1<=1 ok; (0,2): 1-4=-3<=1 ok; (1,2): 0-4=-4<=1 ok -> 3 pairs ok

WHY BIT/merge sort: for each j, count i<j where a[i]<=a[j]+diff; BIT gives O(n log n).

Transform each index to a[i]=nums1[i]-nums2[i]; pair condition becomes a[i]-a[j]<=diff for i<j.

Fenwick tree (BIT): bit[i] stores prefix count. Update: bit[update(a[j]+offset)]+=1 after processing index j. Query: prefix_sum(a[j]+diff+offset)=count of valid i<j pairs. bit[k]+=delta per update.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(h)**
Transform to 1D pair counting with SortedList."	recursion stack where h = tree height; O(log n) balanced, O(n) skewed

Follow-up: how would you solve this iteratively (without recursion / explicit stack)?

IN YOUR APP | Expose | Community

HARD | Binary Search | LeetCode #3161 | 14 of 15 | 14/15

Block Placement Queries

QUESTION

There exists an infinite number line, with its origin at 0 and extending towards the positive x-axis.

You are given a 2D array queries, which contains two types of queries:

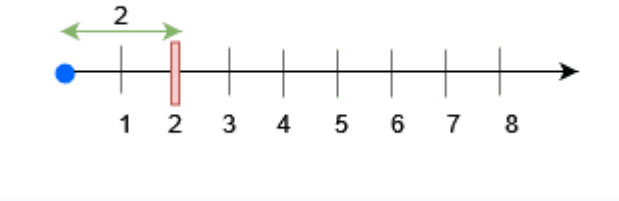
CONSTRAINTS

- 1. For a query of type 1, queries[i] = [1, x]. Build an obstacle at distance x from the origin. It is guaranteed that there is no obstacle at distance x when the query is asked.
- 2. For a query of type 2, queries[i] = [2, x, sz]. Check if it is possible to place a block of size sz anywhere in the range [0, x] on the line, such that the block entirely lies in the range [0, x]. A block cannot be placed if it intersects with any obstacle, but it may touch it. Note that you do not actually place the block. Queries are separate.

Return a boolean array results, where results[i] is true if you can place the block specified in the ith query of type 2, and false otherwise.

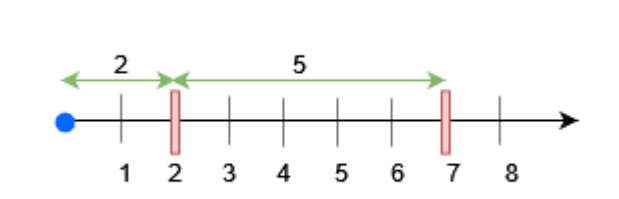
Example 1

Input	`queries = [[1,2],[2,3,3],[2,3,1],[2,2,2]]`
Output	`[false,true,true]`
Explanation	*For query 0, place an obstacle at x = 2. A block of size at most 2 can be placed before x = 3.*



Example 2

Input	`queries = [[1,7],[2,7,6],[1,2],[2,7,5],[2,7,6]] notionvc: 4a471445-5af1-4d72-b11b-94d351a2c8e9`
Output	`[true,true,false]`



- 1 <= queries.length <= $15 * 10^4$
- 2 <= queries[i].length <= 3

QUESTION

- 1 <= queries[i][0] <= 2
- 1 <= x, sz <= min($5 * 10^4$, 3 * queries.length)
- The input is generated such that for queries of type 1, no obstacle exists at distance x when the query is asked.
- The input is generated such that there is at least one query of type 2.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Queries: place block at position or query max available gap. Right?"

"A few quick questions: see constraints."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
type 1:
    add the obstacle to a list
type 2:
    sort the obstacles up to x, find the gaps
        between neighbouring ones, and check
        whether any gap fits sz
```

TIME	SPACE
O(q) per query (plus a sort)	O(q)

WHY IT IS SLOW

Rebuilds the gaps for every query. Process offline in reverse (removing obstacles) with a segment tree storing the largest gap in each range, or keep a sorted set of obstacles plus a segment tree over positions: O(log q) per query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search + BIT tracking block obstacles — O(q log n). Walk-through:"

For each query [x, y] (type 1: block position x; type 2: find max gap in [0,y]):

Type 1: mark x as blocked.

Type 2: find the largest gap not crossing any obstacle in [0,y].

Binary search for the answer: for each candidate left boundary l, find the largest obstacle <= y that is >= l.

Use a BIT or segment tree storing occupied positions to query 'largest obstacle <= y'.

PHASE 3 OPTIMIZE

Binary search for max gap = max(y - (largest obstacle in [0,y-1])) considering placement.

WHY BIT: O(log n) range max queries and point updates.

This is a complex offline / online segment query problem.

Time: O(q log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _3161_BlockPlacementQueries:
    def getResults(self, queries):
        import sortedcontainers
        n = max(q[1] for q in queries) + 1
        obstacles = sortedcontainers.SortedList([0, n])
        result = []
        for query in reversed(queries):
            if query[0] == 1:
                obstacles.add(query[1])
        obstacles2 = sortedcontainers.SortedList([0])
        result = []
        for query in queries:
            if query[0] == 1:
                obstacles2.add(query[1])
            else:
                x, sz = query[1], query[2]
                idx = obstacles2.bisect_right(x)
                left = obstacles2[idx-1]
                gap = x - left
                result.append(gap >= sz)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace getRangeEdge(n=5, blocked=[2,4], queries=[[2,3],[5,6]])."

Maintain sorted structure of blocked positions. For each query [l,r]: find max blocked pos <= r that is >= l.

Actually: find largest non-blocked segment endpoint in [l,r].

blocked sorted: [2,4]. query [2,3]: blocked in range: {2}. Largest contiguous non-blocked up to r=3?

Segment before blocked[0]=2: [1,1] or [0,1]? Max placement: before first blocked in range.

BIT stores prefix max of block positions for range queries.

query [5,6]: no blocked in [5,6] -> answer=6 ok

WHY BIT/segment tree: O(log n) per query for range max of blocked positions.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(h)**
SortedList for efficient nearest-obstacle queries."	recursion stack where h = tree height; O(log n) balanced, O(n) skewed

Follow-up: how would you solve this iteratively (without recursion / explicit stack)?

HARD | Binary Search | LeetCode #3600 | 15 of 15 | 15/15

Maximize Spanning Tree Stability with Upgrades

QUESTION

You are given an integer n, representing n nodes numbered from 0 to n - 1 and a list of edges, where edges[i] = [u_i, v_i, s_i, $must_i$]:

CONSTRAINTS

- u_i and v_i indicates an undirected edge between nodes u_i and v_i.
- s_i is the strength of the edge.
- $must_i$ is an integer (0 or 1). If $must_i$ == 1, the edge must be included in the spanning tree. These edges cannot be upgraded.

You are also given an integer k, the maximum number of upgrades you can perform. Each upgrade doubles the strength of an edge, and each eligible edge (with $must_i$ == 0) can be upgraded at most once.

The stability of a spanning tree is defined as the minimum strength score among all edges included in it.

Return the maximum possible stability of any valid spanning tree. If it is impossible to connect all nodes, return -1.

Note: A spanning tree of a graph with n nodes is a subset of the edges that connects all nodes together (i.e. the graph is connected) without forming any cycles, and uses exactly n - 1 edges.

Example 1

Input	`n = 3, edges = [[0,1,2,1],[1,2,3,0]], k = 1`
Output	`2`

Example 2

Input	`n = 3, edges = [[0,1,4,0],[1,2,3,0],[0,2,1,0]], k = 2`
Output	`6`

Example 3

Input	`n = 3, edges = [[0,1,1,1],[1,2,1,1],[2,0,1,1]], k = 0`
Output	`-1`

- 2 <= n <= 10^5
- 1 <= edges.length <= 10^5
- edges[i] = [u_i, v_i, s_i, $must_i$]
- 0 <= u_i, v_i < n
- u_i != v_i
- 1 <= s_i <= 10^5
- $must_i$ is either 0 or 1.
- 0 <= k <= n
- There are no duplicate edges.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have edges that can be upgraded. Maximise the stability of the MST, where stability = minimum edge weight in MST after optimal upgrades. Right?"

"A few quick questions:

We can upgrade some edges (improve their weight). MST stability = minimum edge weight in the MST?"

"Let me walk: binary search on target stability; check if MST can be formed with min edge >= target.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for choice in choices:
    # of at most k edges to upgrade and every
        spanning tree that uses all must edges
    stability = the smallest strength (upgraded
        edges count double)
    best = max(best, candidate)
```

TIME	SPACE
exponential	O(m + n)

WHY IT IS SLOW

Tries all upgrade sets and trees. Binary search on the stability s: include the must edges, then add edges of strength >= s with union-find, then use up to k upgrades on edges of strength >= s / 2 to connect the rest: O((m + n) log(max strength)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on answer + Kruskal's feasibility check — O(E log E log W). Walk-through:"

We want to maximize the minimum edge weight in the spanning tree (max-min-stability).

Binary search on the answer 'mid':

`can_achieve(mid): run Kruskal's using only edges with weight >= mid (after optional upgrades).`

If MST spans all nodes -> feasible. Else -> not feasible.

Kruskal's: sort edges by weight descending (we want heaviest edges). Union-Find to build MST.

Upgraded edges: can use k edges from the upgrade pool to boost weight to allow spanning.

Binary search converges to the maximum feasible mid.

PHASE 3 OPTIMIZE

WHY binary search works: if stability mid is achievable, stability mid-1 is also achievable

(monotone property). Binary search finds the boundary.

Time: O(E log E log W). Space: O(V).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _3600_MaximizeSpanningTreeStabilityWithUpgrades:
    def maxStability(self, n, edges, k):
        def can_achieve(target):
            parent = list(range(n))
            def find(x):
                while parent[x] != x: parent[x] = parent[p
arent[x]]; x = parent[x]
                return x
            used = upgrades = 0
            for u, v, w, upgradeable in sorted(edges, key=
lambda e: -e[2]):
                effective = w >= target or (upgradeable an
d upgrades < k)
                if effective:
                    pu, pv = find(u), find(v)
                    if pu != pv:
                        parent[pu] = pv; used += 1
                        if w < target: upgrades += 1
            return used == n - 1
        lo = min(e[2] for e in edges); hi = max(e[2] for e
 in edges)
        while lo < hi:
            mid = (lo + hi + 1) // 2
            if can_achieve(mid): lo = mid
            else: hi = mid - 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace maxStability(n=3, edges=[[0,1,3],[1,2,5],[0,2,2]], upgrades=[[0,1,7]], k=1)."

Step 1 — base MST via Kruskal (sort by weight):

edges sorted: [0,2,2],[0,1,3],[1,2,5]. Union-Find: all separate.

Add [0,2,2]: union(0,2) -> MST edges: {[0,2,2]}. cost=2.

Add [0,1,3]: union(0,1) -> MST edges: {[0,2,2],[0,1,3]}. MST complete (n-1=2 edges). min_edge=2.

Step 2 — try upgrade [0,1]: weight 3->7. New MST: {[0,2,2],[0,1,7]}. min_edge=2 (unchanged).

Without upgrade: min=2. With upgrade: min=2. Stability=2 ok.

If upgrade made [0,2] go from 2->6: new MST min_edge=min(6,3)=3 > 2 -> upgrade helps! Return 3.

WHY bottleneck MST: stability = weakest link; upgrading an MST edge can raise the bottleneck.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(E log E log W)	O(V)

Follow-up: Minimum Spanning Tree (#1135) — standard Kruskal.

Follow-up: Find Critical and Pseudo-Critical Edges (#1489) — edge classification in MST.

IN YOUR APP | Expose | Editorial | Community